

AgentBridge AI Studio

Documentation of Understanding

AI Agent Orchestration Platform - Challenge Submission

Project Overview

This document explains the AI Agent Orchestration Platform implementation, the architecture choices, and how the solution satisfies each evaluation criterion end to end.

Developed by Md Azharuddin Ansari

[Follow me on LinkedIn: linkedin.com/in/azhar786](https://www.linkedin.com/in/azhar786)

Area	What I implemented
Runtime	LLM-only custom async runtime using Ollama as local open-source model backend
Agents	Configurable agents with prompt, role, tools, channels, memory, skills, schedules, guardrails, and limits
Workflow	Visual templates with conditions, feedback loops, inter-agent messaging, and persisted execution history
Channel	WhatsApp Cloud API connector with token, phone number ID, recipient, webhook, and auto-send support

1. Executive Summary - My Understanding

My understanding of the challenge is that the team is not looking for a simple chatbot or a UI mockup. The requirement is to build a working AI agent orchestration platform where users can visually create agents, configure how they behave, attach tools and channels, and then connect these agents into collaborative workflows that execute real tasks autonomously.

In my implementation, I treated the problem as a platform engineering challenge: the UI is only one layer, while the real value is in the runtime, persistence, tool execution, message history, channel integration, and monitoring. The platform should feel smooth to use, but the backend must prove that agents are actually running through a real orchestration flow.

Core interpretation

The best solution is not one big assistant. It is a configurable platform where multiple specialized agents communicate, execute tools, persist their work, and expose the result through a visual UI and an external messaging channel.

2. Problem Statement in My Words

I understood the problem statement as: build a platform that allows a non-developer or admin user to create AI agents, define their personality and behavior, assign tools and limits, connect them through a visual workflow, and allow a human to interact with at least one agent through an external channel such as WhatsApp.

The platform must show that agent execution is real. That means when a workflow is started, agents should not simply display static sample text. They should execute through an actual runtime, call tools, pass messages to other agents, write history to a database, and produce output that can be monitored and audited.

3. Requirement-by-Requirement Understanding

Requirement	My understanding	How I addressed it
Agent CRUD	Users must be able to create, read, update, and delete agents with name, role, prompt, model, tools, and channels.	Implemented through REST APIs and the web UI agent registry.
Agent configuration	Agents should support schedules, memory, skills, interaction rules, guardrails, and limits.	Implemented as agent fields persisted in SQLite and rendered in the UI.
Real runtime	Agent logic must actually run and execute tools; it cannot be just frontend mock data.	Implemented through an LLM-only custom async runtime backed by Ollama.
Async communication	Agents must communicate asynchronously with each other.	Implemented through async workflow execution and persisted inter-agent bus messages.
Visual workflow builder	Users need to visually connect agents with conditions and feedback loops.	Implemented through workflow templates, node/edge structure,

		conditions, and feedback loop support.
2+ templates	The platform should start with reusable workflow templates.	Seeded with multiple challenge-specific templates.
External channel	At least one agent must be reachable through WhatsApp, Telegram, or Slack.	Implemented WhatsApp Cloud API connector and webhook endpoints.
Monitoring	Real-time logs, messages, token/cost tracking should be visible.	Implemented Monitoring tab, logs, metrics, message history, and token estimates.
Persistence	History and configuration must survive requests.	Implemented SQLite persistence for agents, workflows, messages, logs, memory, config, and metrics.
Tests/docs	Critical paths and architecture must be documented.	Implemented tests plus README and supporting docs.

4. My Solution Concept

I built AgentBridge AI Studio as a local-first agent orchestration platform. The goal was to keep the project easy to run while still showing a real architecture. I avoided depending on paid cloud LLM APIs and used Ollama so the agents can run with an open-source local model. This also supports the requirement that the project should run fully local with simple setup.

The system is split into four clean layers: frontend UI, FastAPI backend, agent runtime, and persistence. The UI manages agents, templates, workflow execution, WhatsApp configuration, and monitoring. The backend exposes REST endpoints. The runtime handles LLM calls, tool execution, guardrails, memory, async message passing, and workflow traversal. SQLite stores all configuration and execution records.

Layer	Responsibility	Why it matters
Web UI	Agent creation, workflow execution, WhatsApp setup, monitoring, and template creation.	Makes the platform usable and visually manageable.
FastAPI Backend	REST APIs for agents, workflows, logs, messages, metrics, and WhatsApp.	Provides clean separation between UI and runtime.
Agent Runtime	Executes LLM calls, tools, memory, guardrails, and inter-agent handoffs.	Proves the system is not a UI mockup.
SQLite Persistence	Stores agents, workflows, messages, logs, memory, metrics, and configuration.	Makes execution auditable and repeatable.
WhatsApp Connector	Sends agent replies and accepts webhook-style inbound messages.	Satisfies external channel interaction.

5. Why I Chose This Stack

- Python was chosen because it is the strongest ecosystem for AI orchestration, LLM integration, async APIs, tool execution, and testing.

- FastAPI was chosen because it is lightweight, modern, async-friendly, easy to document, and easy to run locally.
- SQLite was chosen because the challenge needs persistence but not complex cloud infrastructure. SQLite keeps setup simple and reproducible.
- A custom async runtime was chosen because the challenge explicitly allows a custom runtime and this gave me full control over agent configuration, tool execution, message bus behavior, and workflow traversal.
- Ollama was chosen for LLM execution because it allows open-source local model usage without requiring paid API keys.
- Vanilla HTML/CSS/JavaScript was used to keep the UI lightweight and easy to run from the same FastAPI server without extra frontend build tooling.

Runtime choice explanation

I intentionally used a custom async runtime because the platform requirement is broader than a single chatbot. The runtime needed to understand configurable agents, tools, memory, limits, guardrails, workflow nodes, conditional edges, feedback loops, and messaging channels.

6. Agent Design

Each agent is treated as a configurable worker rather than a hardcoded function. The agent has a role, personality/system prompt, selected model, tool list, channels, memory setting, skills, interaction rules, schedules, and guardrails. This design directly maps to the challenge requirement that users must configure how agents behave and operate.

Agent	Purpose	Primary capabilities
Orchestrator Agent	Breaks down the user request and coordinates specialist agents.	Planning, routing, task decomposition, workflow control.
Task Analyst Agent	Analyzes the user problem and extracts important context.	Summarization, sentiment/risk check, current-time awareness.
Action Planner Agent	Creates clear owner-level action items.	Action item extraction, prioritization, manager-friendly next steps.
QA Reviewer Agent	Reviews final output for completeness and safety.	Compliance guard, sentiment review, quality check.
WhatsApp Messenger Agent	Prepares and sends channel-ready responses.	WhatsApp-ready message formatting and auto-send.

7. Tool Execution Design

The tools are implemented separately from the LLM so the runtime can decide which tools to run, capture their outputs, and pass the results back into the LLM as grounded context. This makes the system more realistic because agents do not only generate text; they also execute functional operations.

Tool	Purpose	Where it helps
summarize_text	Creates concise summaries from long	Analyst and summarizer steps.

	user input.	
extract_action_items	Extracts actionable next steps from the task.	Action Planner output and UI action table.
sentiment_check	Classifies tone or risk signal.	Review, escalation, and monitoring.
compliance_guard	Detects credential-like or unsafe terms.	Guardrails before sending channel messages.
calculator	Executes safe numeric calculations.	Useful for tasks with numbers and estimates.
current_time	Adds runtime timestamp context.	Logs, updates, and workflow traceability.

8. Workflow Design

The workflow model is built around nodes and edges. Each node references an agent, and each edge defines how control moves from one agent to another. Edges can be unconditional, conditional, or configured as feedback loops. This aligns with the requirement for a visual workflow builder with conditions and feedback loops.

When a workflow runs, the platform creates a conversation ID, stores the user message, starts the entry agent, executes tools, calls the LLM, persists the agent output, sends an async inter-agent message to the next agent, and continues until the workflow completes or reaches its configured step limit.

Workflow template	Purpose	Agents involved
Business Update Workflow	Summarizes an operational issue, creates action items, reviews quality, and prepares a manager update.	Orchestrator, Analyst, Action Planner, Reviewer.
Support Triage Workflow	Classifies an inbound request, checks risk/tone, and produces a response plan.	Orchestrator, Analyst, Action Planner.
WhatsApp Response Workflow	Runs a workflow from a channel message and sends the final response to WhatsApp.	Orchestrator, Analyst, Planner, WhatsApp Messenger.

9. External Messaging Channel - WhatsApp

For the external channel requirement, I implemented WhatsApp Cloud API integration because it is a real business messaging channel and fits the requirement that a human should be able to interact with at least one agent conversationally through WhatsApp, Telegram, or Slack.

The UI provides a WhatsApp Connector Setup section where the user can enter the required channel details: access token, phone number ID, WhatsApp Business Account ID, verify token, graph API version, recipient mobile number, app secret, default workflow ID, and auto-send setting. Secrets are masked after saving.

The outbound flow is: user triggers a workflow from the UI or WhatsApp channel tab; the agent workflow runs; the final agent response is generated by the LLM; if auto-send is enabled, the backend sends that final response to the configured WhatsApp recipient.

WhatsApp field	Why it is needed
Access Token	Authorizes the backend to call Meta WhatsApp Cloud API.
Phone Number ID	Identifies the WhatsApp sender number used by Cloud API.
Recipient Mobile	Target number that receives the agent response.
Graph API Version	Controls the Meta Graph API version used by the request.
Verify Token	Used for webhook verification when receiving messages.
Default Workflow ID	Defines which workflow should run for inbound channel messages.

10. Persistence and Monitoring

I interpreted persistence as a core requirement, not an optional implementation detail. If agents are collaborating, their messages and logs must be visible and auditable. Therefore, the platform persists agents, workflow templates, messages, logs, memory, WhatsApp configuration, and metrics in SQLite.

Data stored	Purpose
Agents	Preserve agent configuration across sessions.
Workflows	Preserve visual templates and execution paths.
Messages	Show user, agent, workflow, and inter-agent communication history.
Logs	Show runtime events such as workflow started, tool executed, LLM request, and workflow completed.
Memory	Allow agents to remember previous context when enabled.
Metrics	Track token estimates, latency, run count, and estimated cost.
WhatsApp config	Store channel settings locally for repeatable testing.

11. UI/UX Understanding

The challenge gives 20% weight to UI/UX and configurability, so I designed the UI as a working operations console rather than a plain form. The UI includes sections for agents, workflows, messaging channels, monitoring, tool registry, and template creation.

- Agent Studio lets users create agents and immediately see status feedback.
- Workflow Builder lets users select templates, create templates, run workflows, and see structured results.
- Messaging Channels provides WhatsApp setup and channel testing.
- Monitoring shows logs, messages, execution status, token estimates, and runtime events.
- The UI uses clean cards, responsive layout, success/error toasts, and readable result sections instead of raw-only JSON.

12. Testing Strategy

I considered the most important test paths to be the ones that prove the system is actually functional: agent creation, workflow execution, message persistence, LLM availability, and channel delivery behavior. The tests are designed around these critical paths.

Test area	What it proves
Health and LLM status	The backend can start and detect the LLM runtime.
Agent creation/listing	Agent CRUD is operational.
Workflow execution	A workflow can run through agents and produce a result.
Message persistence	Conversation history is written and retrievable.
Channel message endpoint	Messaging channel can trigger workflow execution.
WhatsApp config/send path	Channel configuration and sending logic are wired correctly.

13. How I Would Explain the End-to-End Flow

1. I start the platform locally and confirm that Ollama is available with the configured model.
2. I open the web UI and show the prebuilt agents with their roles, tools, channels, memory, and guardrails.
3. I open the Workflow Builder and select a prebuilt workflow template.
4. I enter a realistic operations request, such as a job failure that was resolved after a memory increase.
5. The Orchestrator Agent plans the task and routes it to the Analyst Agent.
6. The Analyst Agent runs relevant tools and generates an LLM-backed analysis.
7. The Action Planner Agent extracts owner-level action items and structured next steps.
8. The QA Reviewer Agent checks completeness and safety.
9. The UI updates the summary, action item table, logs, metrics, and message history.
10. If WhatsApp auto-send is enabled, the final response is sent to the configured WhatsApp number.

14. How This Meets the Evaluation Criteria

Evaluation area	My focus	Evidence in the implementation
Working end-to-end execution	Make the platform run locally and complete a workflow with multiple agents.	FastAPI app, Ollama LLM, workflow endpoint, UI run button, persisted result.
Architecture and code quality	Separate UI, API, runtime, tools, persistence, and integrations.	Project folders, modular runtime, SQLite database layer, tests, docs.
UI/UX and configurability	Make the platform visually manageable and configurable.	Agent forms, template creation, workflow result cards, WhatsApp setup UI, monitoring.
Documentation	Explain setup, architecture, runtime choice, and extension paths.	README and documentation files.

15. Tradeoffs and Design Decisions

- I used SQLite instead of Postgres to keep the setup local and simple. This is sufficient for the challenge, and the DB layer can later be swapped for Postgres.
- I used Ollama instead of paid LLM APIs so the project can be demonstrated without exposing API keys or requiring cloud billing.
- I used a custom runtime instead of LangGraph/CrewAI because the challenge allows a custom runtime, and it gave full control over how agents, tools, memory, channels, and workflow conditions are represented.
- I kept the workflow templates focused on the challenge requirements instead of adding unrelated RAG or database capabilities.
- WhatsApp sending depends on Meta Cloud API credentials, so the project provides clear UI fields for token, phone number ID, recipient number, and webhook verification token.

16. Future Improvements

- Add drag-and-drop node editing for workflow creation instead of only form-based template creation.
- Add per-agent model selection and model performance comparison.
- Add durable background workers for long-running workflows.
- Add role-based access control for admin vs. operator users.
- Add formal cost tracking if a cloud LLM provider is plugged in later.
- Add production-grade encrypted secret storage for WhatsApp tokens.
- Add richer schedule execution so agents can run periodic tasks automatically.

17. Closing Summary

In summary, my goal was to build a platform that proves end-to-end agent orchestration, not just a chatbot. I focused on giving users the ability to create and configure agents, connect them into workflows, execute real tools, persist every interaction, monitor the runtime, and reach the system through WhatsApp. I also kept the system local-first using FastAPI, SQLite, and Ollama so it can be evaluated easily without depending on paid APIs.

The strongest part of this solution is that the agent runtime is explicit and explainable. Each workflow step can be traced: which agent ran, which tool executed, what the LLM generated, which messages were passed, what logs were stored, and whether a WhatsApp message was sent. This makes the project easy to walk through technically and easy to evaluate against the challenge requirements.

Appendix A - Local Run Commands

```
ollama pull llama3.2:3b
py -3.12 -m venv .venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```



```
AgentBridge AI Studio | Documentation of Understanding
.\venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python scripts\seed_workspace.py
$env:OLLAMA_URL="http://127.0.0.1:11434"
$env:OLLAMA_MODEL="llama3.2:3b"
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Appendix B - Suggested Evaluation Prompt

Workflow test prompt

Summarize this EMR project delay issue and create action items. The job failed due to memory issue, memory was increased, and final output generated successfully in S3. Prepare a manager-friendly update and include owner-level next steps.